

```
print("Hello, World!")
```



LINGUAGEM DE PROGRAMAÇÃO PYTHON

AULA 06

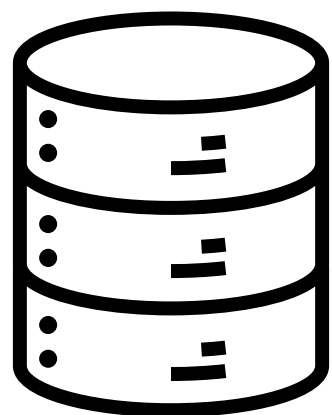
**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

ANTERIORMENTE

- **Break e Continue;**
- **Apresentação de funções;**
- **Exemplo de utilização;**
- **Estrutura de uma função em Python;**
- **Parâmetros;**
- **Escopo de variáveis.**

PRÓXIMOS PASSOS

- Estruturas de dados nativas do Python;
- Listas, tuplas, dicionários, conjuntos;
- Métodos mais comuns;
- Exercícios.



Estruturas de Dados

O que são estruturas de dados em Python?

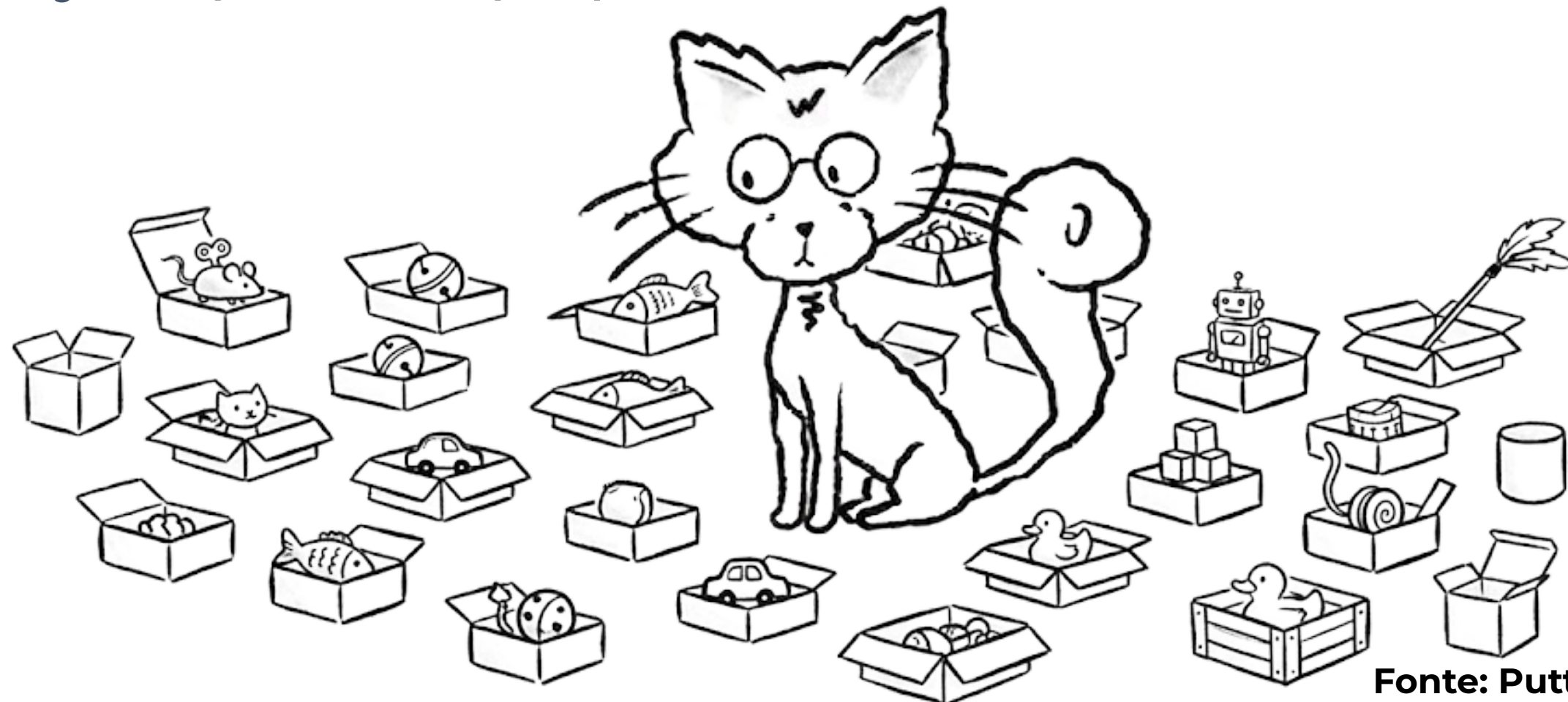
Elas determinam como a informação é armazenada, otimizando o desempenho, o acesso e a manipulação dos dados no código.

Estruturas Nativas:

- Listas,
- Tuplas,
- Dicionários,
- Conjuntos.

Por que usar estruturas de dados?

- **Gestão de Dados:** Essencial para organizar e buscar informações.
- **Desempenho:** Otimizar o tempo de execução e o uso de memória.
- **Qualidade:** Promove um código mais limpo, e de fácil manutenção pela equipe.



Fonte: Putten e Putten (2026)

**Por que não usar apenas uma
estrutura para tudo?**

Escolhendo a Estrutura Certa

Cada estrutura de dados tem características específicas que a tornam mais adequada para determinadas situações. Pergunte-se antes de escolher:

- Os dados vão mudar?
- A ordem importa?
- Posso ter dados repetidos?

O que você ganha ao acertar:

- Melhor desempenho e uso de memória.
- Código legível e fácil de manter.
- Flexibilidade na resolução de problemas.



Fonte: Putten e Putten (2026)

Mutáveis vs. Imutáveis

Característica	Estruturas Mutáveis	Estruturas Imutáveis
Comportamento	Permitem modificações livres após a criação.	Não podem ser alteradas; permanecem constantes.
Principal Benefício	Flexibilidade: Ideais para dados dinâmicos que mudam constantemente.	Segurança e Estabilidade: Garantem previsibilidade para dados fixos.
Exemplos em Python	Listas, Conjuntos e Dicionários.	Tuplas.

Estruturas Ordenadas vs. Não Ordenadas

Característica	Estruturas Ordenadas	Estruturas Não Ordenadas
Comportamento	Mantêm a sequência exata de inserção dos elementos.	Não mantêm a ordem de inserção dos dados.
Principal Benefício	Essencial para aplicações onde a sequência lógica importa.	Vantajosas para otimizar a velocidade de buscas e inserções.
Exemplos em Python	Listas, Tuplas e Dicionários*	Conjuntos

*Dicionários mantêm a ordem a partir do Python 3.7

Aceitam dados duplicados?

Característica	Permitem Duplicatas	Exigem Exclusividade (Sem Duplicatas)
Comportamento	Guardam todas as entradas, mesmo repetidas.	Filtram repetições para garantir dados únicos.
Estruturas	Listas e Tuplas	Conjuntos e Dicionários* (chaves)

*Em dicionários, as chaves são únicas, mas os valores podem se repetir.

Listas



0	10
1	20
2	30
3	40

O que é uma Lista?



Ordenada: Os elementos mantêm a ordem de inserção.



Mutável: Permite adicionar, remover e alterar itens.



Permite Duplicatas: Guarda valores repetidos.



Sintaxe: Representada por colchetes [] .



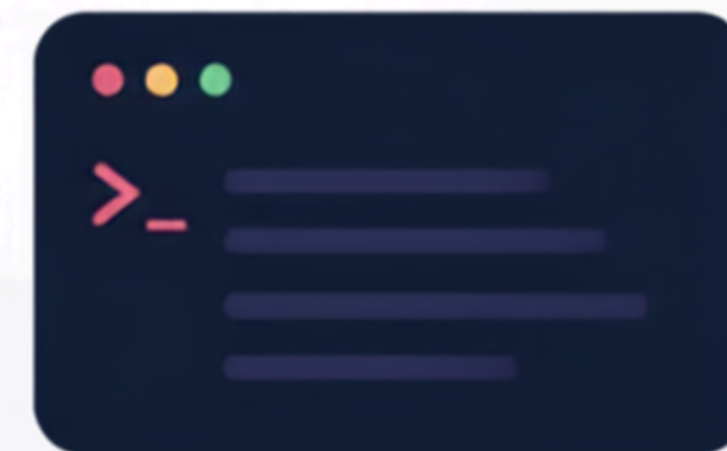
Flexibilidade: Aceita qualquer tipo de dado (números, textos, outras listas).



Acesso: Feito por índices numéricos (iniciando em 0).



Possui muitos métodos que permitem realizar operações (semelhante as strings).



Listas

```
# Uma lista vazia
```

```
uma_das_listas = []
```

```
# Uma lista de brinquedos de gatos
```

```
brinquedos_gatos = ["pena", "bolinha", "apontador laser", "arranhador"]
```

```
print("Primeiro brinquedo:", brinquedos_gatos[0])
```

```
print("Segundo brinquedo:", brinquedos_gatos[1])
```

```
print("Último brinquedo:", brinquedos_gatos[-1])
```

```
Primeiro brinquedo: pena
```

```
Segundo brinquedo: bolinha
```

```
Último brinquedo: arranhador
```

Operações Básicas

```
# Modificando itens da lista
brinquedos_gatos[1] = "ratinho"
print("Lista de brinquedos atualizada:", brinquedos_gatos, sep="\n")
```

Lista de brinquedos atualizada:

```
['pena', 'ratinho', 'apontador laser', 'arranhador', 'caixa de papelão']
```

```
# Adicionando novos elementos
```

```
brinquedos_gatos.append("bolinha") # adiciona ao final
```

```
brinquedos_gatos.insert(0, "varinha") # adiciona na posição 0
```

```
print("Lista de brinquedos atualizada:", *brinquedos_gatos, sep="\n - ")
```

Lista de brinquedos atualizada:

- varinha
- pena
- ratinho
- apontador laser
- arranhador
- caixa de papelão
- bolinha

Operações Básicas

Obrigada!



Fonte: Putten e Putten (2026)

```
# Removendo elementos
```

```
brinquedos_gatos.remove("bolinha") # remove a primeira ocorrência
```

```
print("Lista de brinquedos atualizada:", brinquedos_gatos)
```

```
brinquedos_gatos.pop() # remove o último elemento
```

```
print("Lista de brinquedos atualizada:", brinquedos_gatos)
```

```
brinquedos_gatos.pop(0) # remove o elemento na posição 0
```

```
print("Lista de brinquedos atualizada:", brinquedos_gatos)
```

```
Lista de brinquedos atualizada: ['varinha', 'pena', 'ratinho', 'apontador laser', 'arranhador']
```

```
Lista de brinquedos atualizada: ['varinha', 'pena', 'ratinho', 'apontador laser', 'arranhador']
```

```
Lista de brinquedos atualizada: ['pena', 'ratinho', 'apontador laser', 'arranhador']
```

Operações Básicas

Inverter a ordem da lista

```
brinquedos_gatos.reverse()
```

```
print("Lista de brinquedos invertida:", brinquedos_gatos, sep="\n")
```

Lista de brinquedos invertida:

```
['arranhador', 'apontador laser', 'ratinho', 'pena']
```

Ordenando a lista

```
brinquedos_gatos.sort() # ordena em ordem alfabética
```

```
print("Lista de brinquedos ordenada:", brinquedos_gatos, sep="\n")
```

Lista de brinquedos ordenada:

```
['apontador laser', 'arranhador', 'pena', 'ratinho']
```

```
# Fatiamento
```

```
print("1º e 2º brinquedo:", brinquedos_gatos[0:2], sep="\n")  
print("3º e 4º brinquedo:", brinquedos_gatos[2:4], sep="\n")  
brinquedos_favoritos = brinquedos_gatos[1:4] # índice 1 ao 3  
print("Brinquedos favoritos:", brinquedos_favoritos, sep="\n")
```

(brinquedos_gatos:[1:4])

1º e 2º brinquedo:

['varinha', 'ratinho']

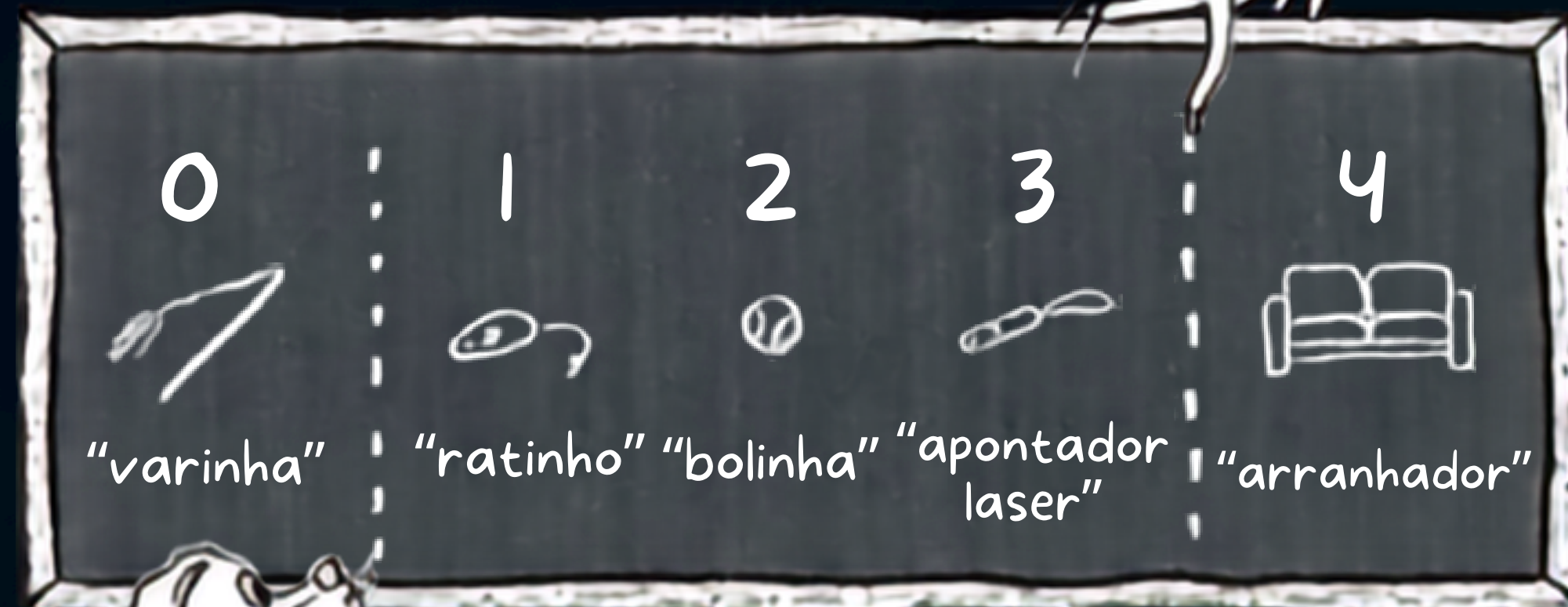
3º e 4º brinquedo:

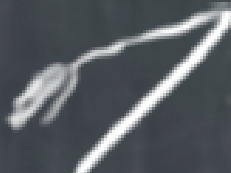
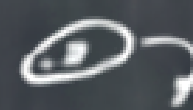
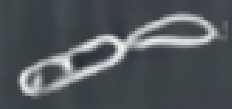
['bolinha', 'apontador laser']

Brinquedos favoritos:

['ratinho', 'bolinha', 'apontador laser']

Esta fatia vai até
o número 4.



0	1	2	3	4
				
"varinha"	"ratinho"	"bolinha"	"apontador laser"	"arranhador"

Fonte: Putten e Putten (2026)

E começa no
número 1,
fácil!"

Mais métodos

Toda essa
conversa sobre
fatiar me deixou
com fome de pizza.

```
# Comprimento da lista
total_brinquedos = len(brinquedos_gatos)
print("Total de brinquedos:", total_brinquedos)
print("Index máximo:", total_brinquedos - 1)
```

Total de brinquedos: 5
Index máximo: 4

```
# Verificando a existência de um item
item_existente = "apontador laser" in brinquedos_gatos
print("Item existente:", item_existente)
```

Item existente: True



Fonte: Putten e Putten (2026)

Usando Condições

```
if "apontador laser" in brinquedos_gatos:    Uau. Que fascinante.  
    print("Hora de brincar!")  
else:  
    print("Hora de descansar!")
```

Hora de brincar!

Removendo com Condição

```
if "caixa de papelão" in brinquedos_gatos:  
    brinquedos_gatos.remove("caixa de papelão")  
    print("A não, ela jogou a caixa fora.")  
else:  
    print("Caixa de papelão não está na lista de brinquedos.")
```

A não, ela jogou a caixa fora.



Fonte: Putten e Putten (2026)

Removendo com Loop

```
brinquedos_gatos.append("varinha")  
print ("Lista de brinquedos:", brinquedos_gatos)
```

Lista de brinquedos: ['ratinho', 'bolinha', 'apontador laser', 'arranhador', 'varinha']

Removendo com Loop

```
for brinquedo in brinquedos_gatos:  
    if brinquedo == "varinha":  
        brinquedos_gatos.remove(brinquedo)  
        print("Removido:", brinquedo)  
    else:  
        print("Não removido:", brinquedo)  
  
print("Lista de brinquedos atualizada:", brinquedos_gatos)
```

Não removido: ratinho

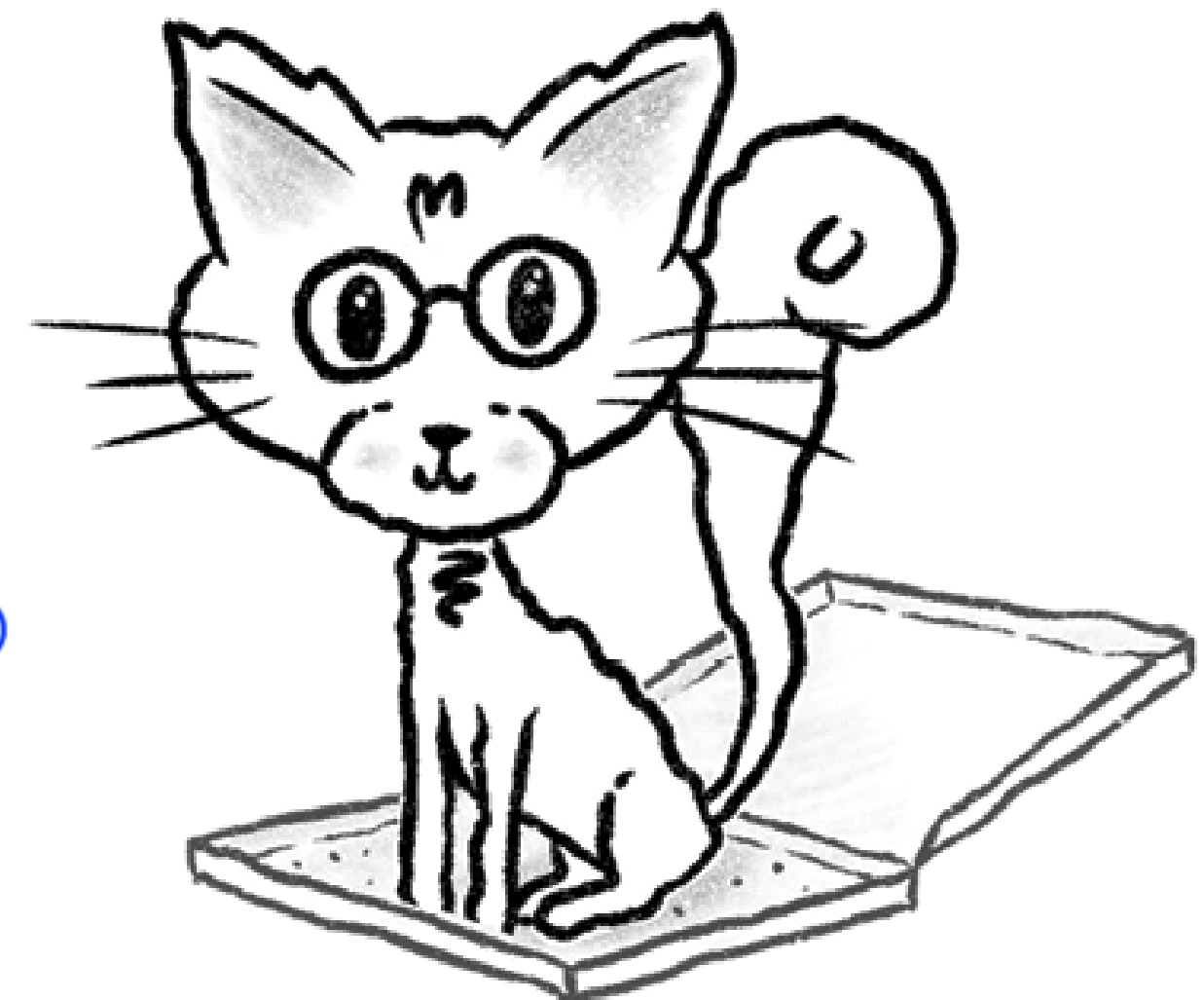
Não removido: bolinha

Não removido: apontador laser

Não removido: arranhador

Removido: varinha

Lista de brinquedos atualizada: ['ratinho', 'bolinha', 'apontador laser', 'arranhador']



Fonte: Putten e Putten (2026)

Métodos

```
# Limpar a lista  
brinquedos_gatos.clear() # limpa a lista  
print("Lista de brinquedos vazia:", brinquedos_gatos)
```

Lista de brinquedos vazia: []



Está na hora de ir à
loja de brinquedos.

Fonte: Putten e Putten (2026)

Atividade Prática 01 Aula 06

Animais Favoritos

Objetivo: Crie uma lista com vários animais para realizar filtros:

Filtros:

1. Mostre em tela seus animais favoritos;
2. Imprima a quantidade de animais da lista e a quantidade de animais favoritos;
3. Mostre qual é o primeiro animal favorito e o último;
4. Remova um dos animais da lista e imprima o resultado;
5. Adicione um ou mais animais na lista;
6. Verifique quantos animais temos agora;
7. Troque a ordem dos animais favoritos de acordo com sua preferência e imprima.

Tuplas

Definição

- Uma tupla é uma coleção ordenada e imutável.
- Representada por parênteses () .
- Pode conter elementos de tipos variados.
- Ideal para dados que não devem ser modificados.
- Os elementos podem ser acessados utilizando índices numéricos que iniciam no 0.

Quando sou
imutável, eles apenas
me chamam de "teimoso".



Fonte: Putten e Putten (2026)

Tuplas

```
# Uma tupla vazia
```

```
uma_das_tuplas = ()
```

```
# Tupla com informações
```

```
info_gatos = ("Batatinha", 12, "Cinza")
```

```
# Não é uma tupla
```

```
nao_e_uma_tupla = ("Batatinha")
```

```
print(type(nao_e_uma_tupla))
```

```
# Maneira correta de criar uma tupla de item único
```

```
tupla_com_um_elemento = ("Batatinha",)
```

```
print(type(tupla_com_um_elemento))
```

```
<class 'str'>
```

```
<class 'tuple'>
```

Tuplas

```
# Tentando alterar o item de uma tupla  
info_gatos[1] = 1
```

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_2882/2600664330.py in <cell line: 0>()  
      1 # Tentando alterar o item de uma tupla  
----> 2 info_gatos[1] = 1
```

TypeError: 'tuple' object does not support item assignment

Quando ser incapaz
de fazer algo
é uma coisa boa?



Desempacotamento

- Uma maneira simplificada de extrair dados de tuplas (e listas).
- Permite extrair mais de 1 valor com apenas 1 linha de código.

Acessar itens da tupla

```
nome, idade, cor = info_gatos  
print("Nome:", nome)  
print("Idade:", idade)  
print("Cor:", cor)
```

Nome: Batatinha
Idade: 12
Cor: Cinza

Acessar itens de uma lista

```
brinquedo01, brinquedo02, *outros_brinquedos = brinquedos_gatos  
print("Brinquedo 01:", brinquedo01)  
print("Brinquedo 02:", brinquedo02)  
print("Outros Brinquedos:", *outros_brinquedos, sep="\n")
```

Brinquedo 01: varinha
Brinquedo 02: ratinho
Outros Brinquedos:
bolinha
apontador laser
arranhador

Operações com tuplas

- Tuplas têm poucos métodos, pois não podem ser alteradas.
- `count(x)`: retorna a quantidade de vezes que o elemento `x` aparece na tupla.
- `index(x)`: retorna o índice da primeira ocorrência de `x` na tupla.

```
# Quantidade e índice de elementos
```

```
print("Quantidade de informações:", len(info_gatos))
```

```
print("Índice do elemento 'Batatinha':", info_gatos.index("Batatinha"))
```

```
print("Quantidade de vezes que 'Batatinha' aparece:", info_gatos.count("Batatinha"))
```

```
Quantidade de informações: 3
```

```
Índice do elemento 'Batatinha': 0
```

```
Quantidade de vezes que 'Batatinha' aparece: 1
```

Atividade Prática 02 Aula 06

O Cardápio Fixo do Axolote (Tuplas)

Objetivo: Como sabemos, as tuplas são coleções imutáveis, sendo o formato ideal para armazenar dados que não devem ser modificados acidentalmente. O axolote de estimação tem uma rotina de alimentação muito restrita!

Atividades:

1. Crie uma tupla contendo três elementos de tipos variados: o horário da refeição (texto), o alimento principal (texto) e a quantidade de vitaminas (número).
2. Tente alterar o alimento principal da tupla para "pizza de calabresa".
3. Delete a linha que causou o erro. Agora, utilize a técnica de desempacotamento de tuplas para extrair esses três valores e salvá-los em três variáveis independentes usando apenas 1 linha de código.
4. Imprima uma frase amigável usando as variáveis desempacotadas.
Por exemplo: "Às 08:00, o axolote come minhocas com 50 vitaminas."

Dicionários

Definição

- Um dicionário é uma coleção ordenada (a partir do Python 3.6), mutável e indexada por chaves.
- Representado por chaves { }.
- Usa o formato **chave: valor**.



Fonte: Putten e Putten (2026)

Dicionários

```
# Um dicionário vazio
```

```
um_dos_dicionarios = {}
```

```
# Dicionário com informações
```

```
atributos_gatos = {  
    "nome": "Batatinha",  
    "idade": 12,  
    "cor": "Cinza",  
    "comida favorita": "Ração"  
}
```

Acessando e modificando dados

- Os itens são acessados e modificados com base na sua chave, que não pode ser alterada.
- Para adicionar um item, basta utilizar uma chave não existente no dicionário.
- Para remover um item podemos utilizar o “del” (obs: Dicionários também possuem o método pop).

Uhh...
O que faço agora?



Fonte: Putten e Putten (2026)

Acessando e modificando dados

```
# Acessando itens
print("Nome:", atributos_gatos["nome"])
print("Idade:", atributos_gatos["idade"])
print("Cor:", atributos_gatos["cor"])
print("Comida favorita:", atributos_gatos["comida favorita"])
```

Nome: Batatinha

Idade: 12

Cor: Cinza

Comida favorita: Ração

Acessando e modificando dados

```
# Tentando acessar uma chave inexistente  
print("Raça:", atributos_gatos["raça"])
```

```
-----  
KeyError                                Traceback (most recent call last)  
/tmp/ipykernel_2882/2126362242.py in <cell line: 0>()  
      1 # Tentando acessar uma chave inexistente  
----> 2 print("Raça:", atributos_gatos["raça"])
```

```
KeyError: 'raça'
```

Uhh...
O que faço agora?



Acessando e modificando dados

```
# Acessando uma chave com get
print("Nome:", atributos_gatos.get("nome"))
print("Raça:", atributos_gatos.get("raça"))
```

Nome: Batatinha
Raça: None

```
# Especificar retorno do get
print("Nome:", atributos_gatos.get("nome", "Não encontrado"))
print("Raça:", atributos_gatos.get("raça", "Não encontrado"))
```

Nome: Batatinha
Raça: Não encontrado

Alterando e Adicionando

```
# Adicionando chave
atributos_gatos["hobby"] = "Soneca"
print("Hobby:", atributos_gatos["hobby"])
#
```

Hobby: Soneca

```
# Alterando chave
atributos_gatos["comida favorita"] = "Caldinho de Frango"
print("Comida favorita:", atributos_gatos["comida favorita"])
#
```

Comida favorita: Caldinho de Frango

Removendo chaves

```
# Removendo chaves
comida_favorita = atributos_gatos.pop("comida favorita")
del atributos_gatos["cor"]
print("Comida favorita removida:", comida_favorita)
print("Cor removida:", atributos_gatos.pop("cor"))
print("Dicionário atualizado:", atributos_gatos)
#
```

Comida favorita removida: Caldinho de Frango

Dicionário atualizado: {'nome': 'Batatinha', 'idade': 12, 'hobby': 'Soneca'}

Métodos úteis

- Dicionários possuem muitos métodos
- Alguns deles são:
 - `keys()`: Retorna as chaves do dicionário
 - `values()`: Retorna os valores
 - `items()`: Retorna pares (chave, valor)
 - `list()`: transforma o valor numa lista

Métodos

Todas as chaves

```
lista_chaves = list(atributos_gatos.keys())  
print("Todas as chaves:", lista_chaves)  
print("Tipo:", type(lista_chaves))
```

Todas as chaves: ['nome', 'idade', 'hobby']
Tipo: <class 'list'>

Todos os valores

```
lista_valores = list(atributos_gatos.values())  
print("Todos os valores:", lista_valores)  
print("Tipo:", type(lista_valores))
```

Todos os valores: ['Batatinha', 12, 'Soneca']
Tipo: <class 'list'>

Métodos

```
# Todas as chaves e valores
lista_chaves_valores = list(atributos_gatos.items())
print("Todas as chaves e valores:", lista_chaves_valores)
print("Tipo:", type(lista_chaves_valores))
print("Primeiro item:", lista_chaves_valores[0])
print("Tipo:", type(lista_chaves_valores[0]))
#
```

```
Todas as chaves e valores: [('nome', 'Batatinha'), ('idade', 12), ('hobby', 'Soneca')]
Tipo: <class 'list'>
Primeiro item: ('nome', 'Batatinha')
Tipo: <class 'tuple'>
```

Atividade Prática 03 Aula 06

Agenda Telefônica (Dicionários)

Objetivo:

1. Crie um dicionário de contatos com alguns nomes (como chaves) e números de telefone (como valores);
2. Adicione um novo contato ao dicionário sem alterá-lo diretamente (na declaração original);
3. Verifique se o contato "Chai" existe nos contatos. Se não, adicione-a com um número de telefone inventado.;
4. Exclua um dos seus contatos do dicionário; use um método que não cause um erro se o contato não existir;
5. Imprima todos os nomes presentes no dicionário.

OBS.: Para cada ação imprima o resultado.

Conjuntos



Fonte: Putten e Putten (2026)

Definição

- Um conjunto (set) é uma coleção não ordenada, mutável e que não permite elementos duplicados.
- Representado por chaves.
- Muito usado para operações matemáticas de união, interseção e diferença.
- Útil para remover duplicatas automaticamente de uma coleção de dados.

Conjuntos

```
# Criando um conjunto a partir de uma lista
print ("Lista de brinquedos:", *brinquedos_gatos, sep="\n - ")
conjunto_de_brinquedos = set(brinquedos_gatos)
print ("Conjunto de brinquedos:", *conjunto_de_brinquedos, sep="\n - ")
```

Lista de brinquedos:

- varinha
- ratinho
- bolinha
- apontador laser
- arranhador
- bolinha

Conjunto de brinquedos:

- ratinho
- varinha
- arranhador
- apontador laser
- bolinha

Funções Básicas de Conjuntos

```
# Adicionando elemento
```

```
conjunto_de_brinquedos.add("bola")
```

```
print ("Conjunto de brinquedos atualizado:", *conjunto_de_brinquedos, sep="\n    ")
```

Conjunto de brinquedos atualizado:

- bola
- ratinho
- varinha
- arranhador
- apontador laser
- bolinha

Funções Básicas de Conjuntos

```
# Removendo elemento
conjunto_de_brinquedos.remove("bola")
print ("Conjunto de brinquedos atualizado:", *conjunto_de_brinquedos, sep="\n - ")
```

Conjunto de brinquedos atualizado:

- ratinho
- varinha
- arranhador
- apontador laser
- bolinha

```
# Removendo elemento que pode não existir
conjunto_de_brinquedos.discard("bola")
print ("Conjunto de brinquedos atualizado:", *conjunto_de_brinquedos, sep="\n - ")
```

Conjunto de brinquedos atualizado:

- ratinho
- varinha
- arranhador
- apontador laser
- bolinha

Funções Matemáticas de Conjuntos

```
# Operação matemática com conjuntos
```

```
caixa_brinquedos_01 = {"bolinha", "varinha de fitas", "apontador laser"}
```

```
caixa_brinquedos_02 = {"apontador laser", "pena", "arranhador"}
```

```
# Verificando a existência de itens
```

```
print("bolinha" in caixa_brinquedos_01)
```

```
print("bolinha" in caixa_brinquedos_02)
```

```
True
```

```
False
```

```
# União do conjuntos
```

```
todos_os_brinquedos = caixa_brinquedos_01.union(caixa_brinquedos_02)
```

```
print("Todos os brinquedos:", todos_os_brinquedos)
```

```
Todos os brinquedos: {'arranhador', 'varinha de fitas', 'apontador laser', 'pena', 'bolinha'}
```

Funções Matemáticas de Conjuntos

Interseção

```
brinquedos_em_comum = caixa_brinquedos_01.intersection(caixa_brinquedos_02)
print("Brinquedos em comum:", brinquedos_em_comum)
```

Brinquedos em comum: {'apontador laser'}

Diferença

```
brinquedos_em_caixa_01 = caixa_brinquedos_01.difference(caixa_brinquedos_02)
brinquedos_em_caixa_02 = caixa_brinquedos_02.difference(caixa_brinquedos_01)
print("Brinquedos em somente em caixa 01:", brinquedos_em_caixa_01)
print("Brinquedos em somente em caixa 02:", brinquedos_em_caixa_02)
```

Brinquedos em somente em caixa 01: {'varinha de fitas', 'bolinha'}

Brinquedos em somente em caixa 02: {'arranhador', 'pena'}

Atividade Prática 04 Aula 06

A Sacola do Clube do Livro (Conjuntos)

Objetivo: Um conjunto (set) é uma estrutura que, por natureza, não permite elementos duplicados, sendo útil para remover duplicatas de coleções de dados automaticamente. Fui à feira de troca de livros com meus amigos e jogamos todos os livros que arrecadamos em uma grande sacola. Acabamos pegando várias cópias do mesmo livro!

Atividades:

1. Crie uma lista inicial com cerca de 6 ou 7 títulos de livros, mas coloque de propósito uns 3 títulos repetidos (por exemplo: "Duna", "O Hobbit", "Duna", "Harry Potter", "O Hobbit").
2. Converta essa sua lista em um Conjunto para forçar o sistema a eliminar todas as cópias duplicadas em um passe de magia.
3. Imprima o conjunto na tela. Note que o resultado final perdeu as repetições e que os itens provavelmente mudaram de lugar, já que os conjuntos não mantêm a ordem de inserção dos elementos!

Atividade Prática 05 Aula 06

Banco de Dados de Biblioteca (Combinação de Tipos)

Objetivo: Imagine que sua biblioteca local peça a você para organizar o catálogo de livros em um dicionário Python. Neste exercício, vamos focar em estruturar e escolher os tipos de dados certos:

1. Crie uma estrutura combinada que armazene informações detalhadas sobre os livros. (Exemplo: O dicionário principal tem o nome do livro como chave, e o valor é outro dicionário contendo o nome do autor, ano de publicação, uma lista de gêneros literários e se ele está disponível ou emprestado)

Estas são as informações que precisamos armazenar sobre cada livro:

- * Título
- * Nome do autor, ano de nascimento e nacionalidade
- * Número de páginas
- * Disponibilidade
- * Membro da biblioteca que o pegou emprestado atualmente
- * Ano de publicação
- * Gêneros
- * Avaliação (1-5)

Atividade Prática 05 Aula 06

Aqui estão alguns títulos de livros para você começar.
Sinta-se à vontade para adicionar alguns livros que você pessoalmente adora.

Título: Python Illustrated

Autores: Zia van Putten, 2023, Holandesa - Maaïke van Putten, 1991, Holandesa - Imke van Putten, 1997, Holandesa

Número de páginas: 432

Ano de publicação: 2026

Gêneros: Informativo, Educação

Avaliação: 5

Disponível: não

Emprestado para: Jacob

Título: Se os gatos desaparecessem do mundo

Autor: Genki Kawamura

Número de páginas: 144

Ano de publicação: 2012

Gêneros: Ficção Literária, Realismo Mágico, Ficção Filosófica

Avaliação: 4.2

Disponível: não

Emprestado para: Chai

Atividade Prática 05 Aula 06

Aqui estão algumas tarefas:

- Imprima uma lista de todos os títulos de livros que a biblioteca possui.
- Verifique se "Romance" é um gênero de "Python Illustrated" e imprima a resposta.
- Imprima a nacionalidade do autor de Jane Eyre.
- Chai devolveu o livro que pegou emprestado. Atualize os valores de "Se os gatos desaparecessem do mundo", de acordo.
- Verifique se "Python Illustrated" está disponível e imprima quem o pegou emprestado caso não esteja.

Qual estrutura de dados você usaria em cada uma das situações abaixo?

- Lista de Compras;
- Registro de Alunos e Notas;
- Datas de eventos históricos;
- Identificar quais alunos de Python também estão matriculados na disciplina de desenvolvimento seguro.

Próximos Passos



**Tratamento de erros;
Revisão;
Lista de Exercícios;
Prova;
Bibliotecas no Python;
Recursividade;**

**exit() ou
dúvidas?**

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

OBRIGADA!

“Special cases aren't special
enough to break the rules.”

The Zen of Python, by Tim Peters



Referências: PUTTEN, Maaïke van; PUTTEN, Imke van. Python Illustrated: Not another boring Python book, learn programming the fun way. Birmingham: Packt Publishing, 2026. [1, 2, 3]

SANTA CRUZ

CENTRO UNIVERSITÁRIO

Para mais
informações
acesso
o **QR CODE**
e saiba mais



 (41)3052-4900

 @unisantacruz

 /UniSantaCruzCtba

 unisantacruztem.com.br